Absolutely. **G1 (Garbage-First) GC** is where JVM garbage collection becomes much more interesting because it combines **generational GC, region-based heap management, concurrent marking, remembered sets, and pause-time goals**.

The most useful mental model is:

> **G1 divides the heap into regions and tries to collect the regions that give the most garbage reclaimed for the amount of pause time available.**

---

1. Why was G1 needed?

Consider the progression:

```
Serial
   ↓
Parallel
   ↓
CMS
   ↓
G1
```

Serial

Uses one GC thread:

```
STW → GC → Resume
```

Parallel

Uses multiple GC threads:

```
STW → GC1 + GC2 + GC3 + GC4 → Resume
```

Better throughput, but pauses can still become long.

CMS

Moves much of marking/sweeping concurrently:

```
STW → concurrent mark → STW remark → concurrent sweep
```

But CMS doesn't compact the old generation concurrently, leading to fragmentation.

G1

G1 combines:

- Region-based heap
- Generational collection
- Concurrent marking
- Remembered sets
- Evacuation/compaction
- Pause-time prediction

---

2. The Biggest Difference: Regions

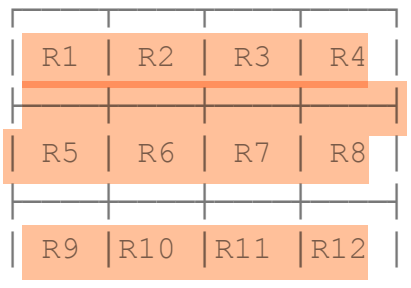
Traditional collectors conceptually divide the heap into:

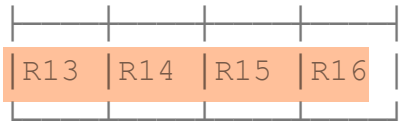
```
Young Generation
----------------
Eden
Survivor

Old Generation
----------------
Old
```

G1 instead divides the **entire heap into many equal-sized regions**.

For example, imagine a simplified 16-region heap:





Each region is the same size.

Real G1 regions are typically **1–32 MB**, chosen based on heap size.

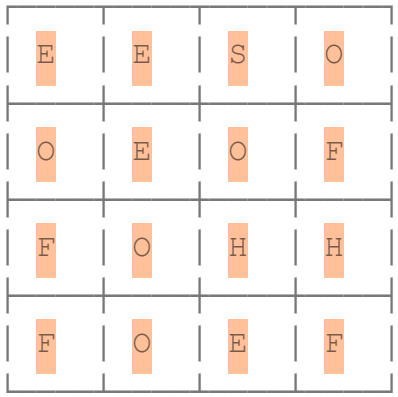
---

3. Regions Have Logical Roles

A region can currently be used as:

```
Eden
Survivor
Old
Humongous
Free
```

For example:




```
E = Eden
S = Survivor
O = Old
H = Humongous
F = Free
```

This is a major conceptual difference.

Young and old aren't necessarily contiguous physical areas.

---

4. Example: Application Starts

Suppose we have:

```
Heap = 16 GB
Region size = 4 MB
```

That gives:

```
4096 regions
```

Initially:

```
Free:
R1 R2 R3 R4 R5 ... R4096
```

Application starts allocating objects.

G1 assigns some regions to Eden:

```
Eden:

R1
R2
R3
R4
```

Objects are allocated inside these regions.

---

5. Eden Fills

Eventually:

```
Eden:

R1 ████████
R2 ████████
```

```
R3 ████████
R4 ████████
```

G1 needs a Young GC.

But here's an important point:

> **G1 Young GC is still Stop-The-World.**

Application threads pause:

```
Application
T1 ──────── X
T2 ──────── X
T3 ──────── X
T4 ──────── X

              ↓

          G1 Young GC
```

---

6. G1 Finds Live Objects

Suppose:

```
R1:
A B C D E

R2:
F G H I

R3:
J K L M

R4:
N O P Q
```

Some objects are dead:

```
R1 → A B C live, D E dead
R2 → F H live, G I dead
R3 → J K L live, M dead
R4 → N live, O P Q dead
```

G1 doesn't simply "clear" the regions.

It **evacuates** live objects.

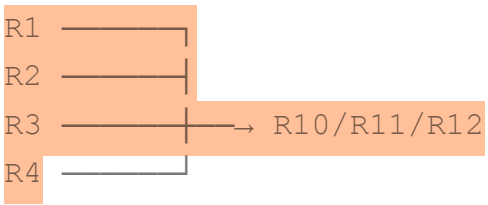
---

7. Evacuation

Suppose G1 chooses free regions:

```
R10
R11
R12
```

Live objects are copied there:



After evacuation:

```
Before:

R1 = A B C D E
R2 = F G H I
R3 = J K L M
R4 = N O P Q

After:

R10 = A B C F
R11 = H J K L
R12 = N
```

The original regions become free:

```
R1 → FREE
R2 → FREE
R3 → FREE
R4 → FREE
```

This is effectively **compaction by evacuation**.

---

8. Why Evacuation Is Powerful

Traditional mark-sweep can produce:

```
A  FREE  B  FREE  C  FREE
```

G1 instead copies survivors together:

```
A B C
```

So free space becomes:

```
FREE FREE FREE
```

Therefore G1 naturally avoids the fragmentation problem that plagued CMS.

---

9. Where Does G1 Get the Roots?

This is where **Remembered Sets (RSet)** become important.

Suppose:

```
Old Region R100
      |
      | reference
```



During Young GC, G1 doesn't want to scan every old region.

Instead, G1 maintains remembered-set information about references coming **into a region**.

Conceptually:

```
R100 ──────────→ R2

R2's Remembered Set:
    R100 → card X
```

Therefore, when collecting R2, G1 knows where external references into R2 may exist.

This is one reason we discussed **card tables and remembered sets** earlier.

---

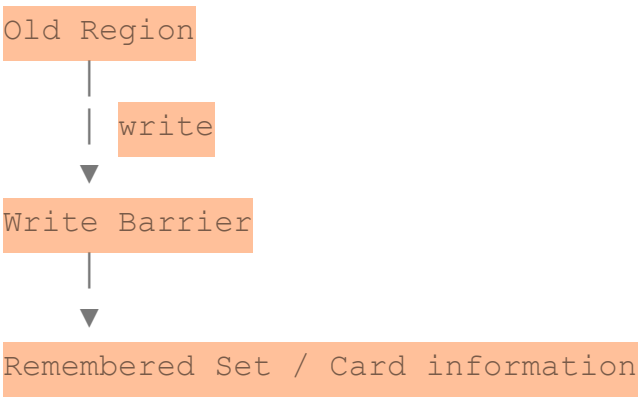
10. The Write Barrier

Suppose the application executes:

```
oldObject.child = youngObject;
```

G1 uses barriers to track this inter-region reference.

Conceptually:



The important idea is:

> **The application helps the GC maintain information about cross-region references.**

---

11. What Makes G1 "Garbage First"?

Now we get to the name.

Suppose we have:

```
Region    Live      Garbage
_____________________________

R1        90%         10%
R2        20%         80%
R3        95%          5%
R4        30%         70%
```

If G1 needs to reclaim memory, which region should it collect?

Obviously:

```
R2
R4
```

They contain lots of garbage.

So:

```
Garbage density:

R2 ████████████████  80%
R4 ██████████████    70%
R1 ███               10%
R3 ▌                  5%
```

G1 prioritizes regions that offer a good **reclamation benefit relative to the predicted pause cost**.

Hence:

> **Garbage First**

---

12. But How Does G1 Know How Much It Can Collect?

This is one of G1's most important features.

Suppose you configure:

```
-XX:MaxGCPauseMillis=200
```

You're effectively telling G1:

> "Try to keep GC pauses around this target."

G1 maintains models of:

- How long evacuation of regions takes
- How much live data each region contains
- How much work GC workers can perform
- How much work can fit into the target pause

It then chooses a **collection set (CSet)**.

---

13. Collection Set

Suppose:

```
Pause target = 200 ms
```

G1 estimates:

```
R1 → 30 ms
R2 → 40 ms
R3 → 50 ms
R4 → 35 ms
R5 → 40 ms
```

It may choose:

```
CSet = R1 + R2 + R3 + R4
```

Estimated pause:

```
30 + 40 + 50 + 35 = 155 ms
```

But adding R5 would push it toward/exceed the target.

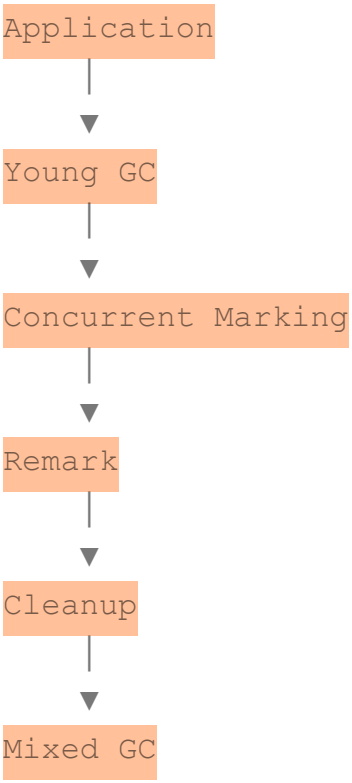
So G1 can stop there.

This is a simplified illustration; actual prediction uses more sophisticated cost models.

---

14. G1's Major Phases

A simplified G1 lifecycle looks like:



Let's understand these.

---

15. Phase 1 — Young GC

When Eden fills:

```
STW
 │
 ▼
Young GC
 │
 ├── scan roots
 ├── process remembered sets
 ├── evacuate live objects
 ├── age survivors
 └── promote some objects
 │
 ▼
Resume application
```

This is similar conceptually to the young collection we saw with Serial/Parallel GC.

---

16. Phase 2 — Concurrent Marking

Eventually G1 needs to determine how much garbage exists throughout the old regions.

It performs concurrent marking.

Conceptually:

```
Application Threads
████████████████████████████████

G1 Concurrent Mark
        ████████████████████
```

The application continues running.

G1 uses **tri-color marking** and a **SATB-style marking barrier** to maintain correctness while references change.

This connects directly to your earlier questions about:

- Tri-color marking
- Write barriers

- STW
- SATB

---

17. Why Does G1 Need SATB?

Imagine at the beginning of marking:

```
A → B
```

GC sees:

```
A → B
```

and considers B reachable.

But while concurrent marking is running, application code changes:

```
A → C
```

Now B is no longer referenced through A.

If GC simply followed the current graph, it could lose track of objects that were reachable at the beginning of marking.

SATB essentially says:

> **Treat the heap as though we're determining liveness based on the snapshot that existed when marking started.**

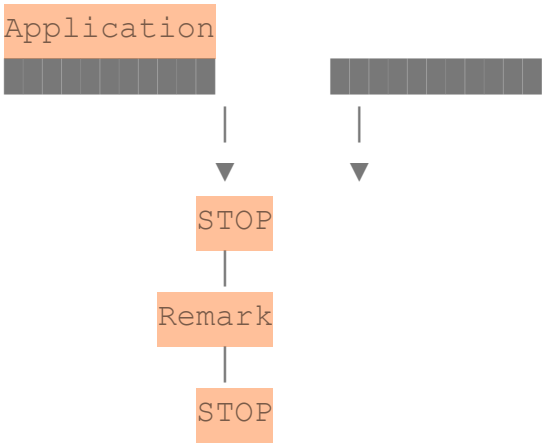
The write barrier records information needed to preserve that snapshot semantics.

---

18. Phase 3 — Remark

Concurrent marking eventually reaches a point where G1 needs to finalize marking.

It briefly stops application threads:



The remark phase processes outstanding marking information and completes the marking result.

This pause is usually much shorter than doing an entire heap collection.

---

19. Phase 4 — Cleanup

G1 determines which regions contain substantial amounts of garbage.

Imagine:

```
Region    Live %
──────────────────
R1          90%
R2          15%
R3          80%
R4          25%
R5          95%
R6          10%
```

G1 now knows:

```
R6 → excellent candidate
R2 → excellent candidate
R4 → good candidate
R3 → less attractive
R5 → poor candidate
```

This information feeds future mixed collections.

---

20. Phase 5 — Mixed GC

This is one of G1's defining features.

A normal Young GC collects:

```
Eden + Survivor
```

A **Mixed GC** collects:

```
Young regions
      +
selected Old regions
```

For example:

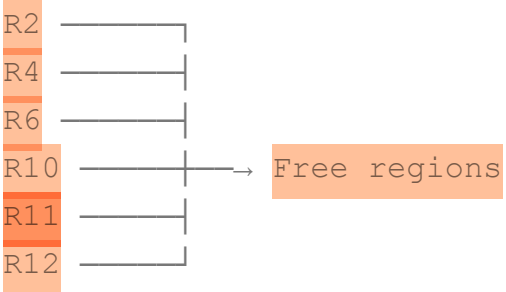
```
Collection Set:

Eden:
R10
R11

Survivor:
R12

Old:
R2
R4
R6
```

G1 evacuates live objects from all of them.



Afterward:

```
R2  → FREE
R4  → FREE
R6  → FREE
R10 → FREE
R11 → FREE
R12 → FREE
```

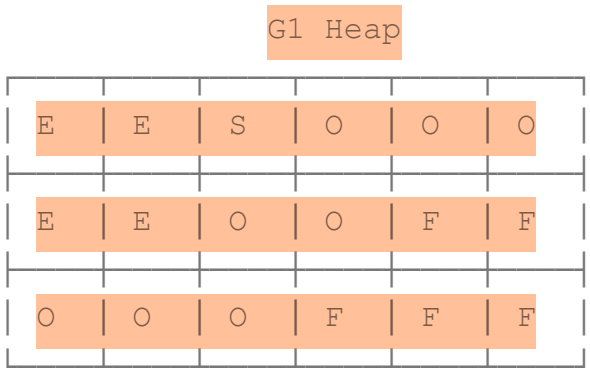
Live objects have been relocated elsewhere.

---

21. Complete Example

Let's put everything together.

Imagine this heap:



Suppose old regions contain:

```
O1 → 90% live
O2 → 20% live
O3 → 10% live
O4 → 70% live
O5 → 15% live
O6 → 95% live
```

G1's marking identifies:

```
High garbage:

O3 → 90% garbage
```

```
O5 → 85% garbage
O2 → 80% garbage
```

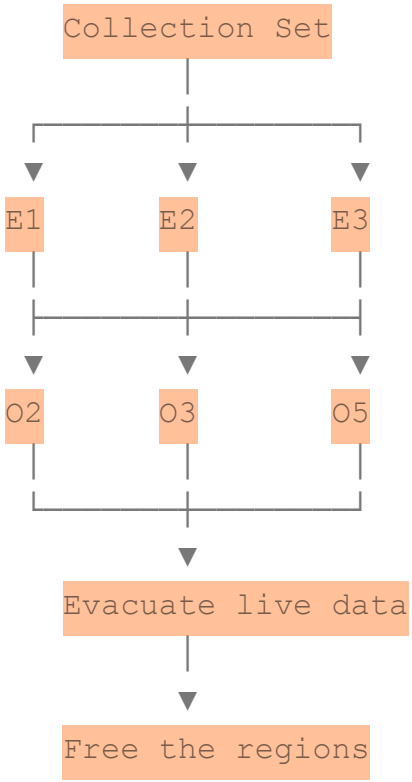
So G1 may eventually choose:

```
Mixed Collection Set:

Young:
E1
E2
E3

Old:
O2
O3
O5
```

During the STW pause:



The result might be:

```
Before:

E1 E2 E3 O2 O3 O5
```

```
After:

FREE FREE FREE FREE FREE FREE
```

while the relatively live-heavy regions remain untouched.

---

22. Humongous Objects

G1 has another interesting concept: **humongous objects**.

Suppose:

```
byte[] data = new byte[20 * 1024 * 1024];
```

If an object is sufficiently large relative to the G1 region size, it may require multiple contiguous regions.

For example:

```
Region size = 4 MB

20 MB object:

┌────┬────┬────┬────┬────┐
│ H  │ H  │ H  │ H  │ H  │
└────┴────┴────┴────┴────┘
```

These are called **humongous regions**.

They're handled differently from ordinary objects and can have important allocation/GC implications.

---

23. G1 vs Parallel GC

This is probably the most important comparison for your interviews.

Parallel GC

```
Eden full
    ↓
STW
    ↓
Multiple GC threads
    ↓
Young collection
    ↓
Resume
```

The goal is:

> **Finish GC quickly.**

G1

```
Eden full
    ↓
STW Young GC
    ↓
Application resumes
    ↓
Concurrent marking
    ↓
Remark
    ↓
Identify garbage-rich regions
    ↓
STW Mixed GC
    ↓
Evacuate selected regions
```

The goal is:

> **Control pause times while efficiently reclaiming memory.**

---

24. Why G1 Doesn't Just Collect the Entire Old Generation

Imagine:

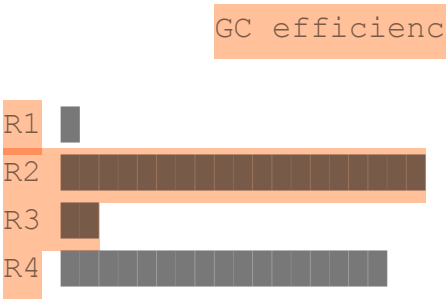
```
Old regions:

R1 → 95% live
R2 → 10% live
R3 → 90% live
R4 → 15% live
```

Collecting R1 would mean moving a huge amount of live data.

Collecting R2 gives much more garbage reclaimed for much less copying.

Therefore:



G1 preferentially chooses R2/R4.

This is the **"Garbage First"** principle.

---

25. The Three Critical G1 Concepts

If you're preparing for a senior/staff-level JVM interview, remember these three:

① Regions

Instead of:

```
[Young][Old]
```

G1 uses:

```
[R][R][R][R][R][R][R]...
```

Regions can have different roles.

---

② Remembered Sets

Instead of scanning the entire heap to find incoming references:

```
R100 ——→ R5
R200 ——→ R5
```

G1 maintains information associated with R5 about those incoming references.

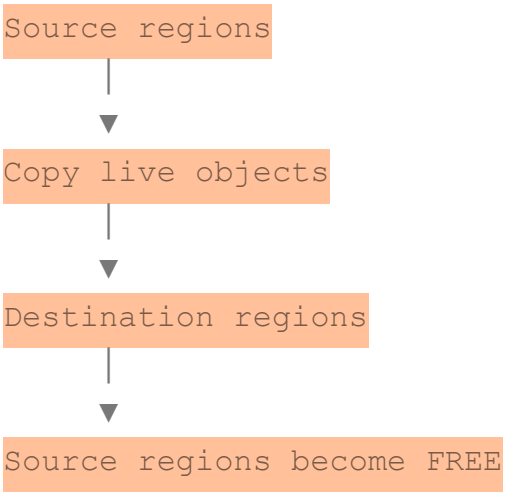
This makes region evacuation practical.

---

③ Evacuation

G1 doesn't primarily solve fragmentation through a traditional global compaction pass.

It:



That's essentially **incremental compaction through evacuation**.

---

26. One Important Correction to a Common Explanation

You may hear:

> "G1 divides the heap into regions instead of having generations."

That's **not correct for modern G1**.

G1 is **generational**.

A better statement is:

> **G1 divides the heap into regions, and those regions are dynamically assigned to Eden, Survivor, Old, or Humongous roles.**

So you can have:

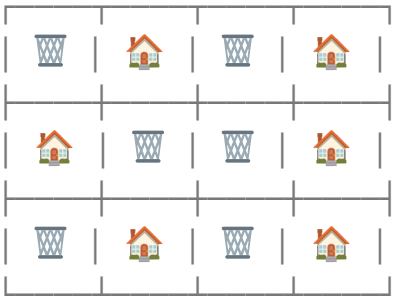
```
        G1 Heap
┌──────────────────────────────────┐
│ Eden regions                     │
│ Survivor regions                 │
│ Old regions                      │
│ Humongous regions                │
│ Free regions                     │
└──────────────────────────────────┘
```

This distinction is valuable in interviews.

---

27. G1 Mental Model

Think of G1 as a **city divided into blocks**.



Instead of shutting down the whole city to clean everything:

> **Clean the blocks containing the most garbage first, and move the useful things elsewhere.**

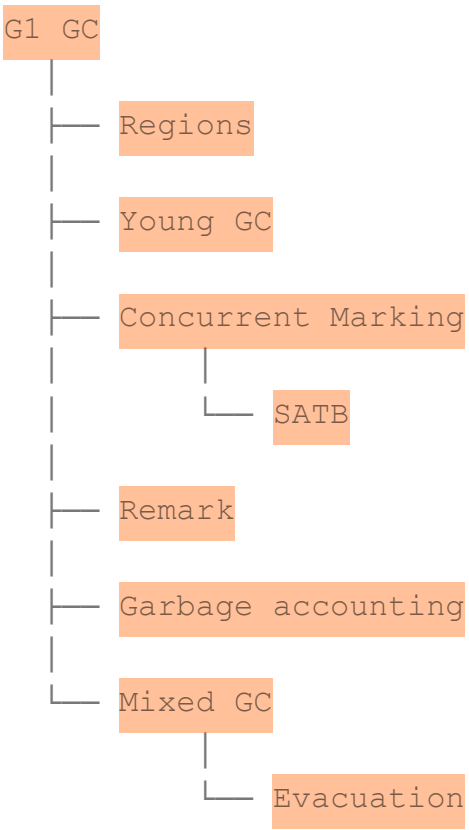
That's essentially what **Garbage-First** means.

🎯 Interview answer

If asked **"Explain G1 GC"**, a strong answer is:

> **"G1 is a region-based, generational garbage collector designed to balance throughput and predictable pause times. It divides the heap into equal-sized regions that are dynamically used as Eden, Survivor, Old, Humongous, or free regions. Young collections are stop-the-world evacuation pauses. G1 concurrently marks the heap to determine the live data in old regions, then selects garbage-rich regions for mixed collections. During those pauses it evacuates live objects from the collection set into other regions, thereby reclaiming and compacting memory. Remembered sets and write barriers allow G1 to efficiently track cross-region references, while its pause-time prediction model helps select how much work can fit within the configured pause target."**

And the conceptual pipeline to remember is:



This is the point where your previous topics—tri-color marking, STW, write barriers, card tables, remembered sets, generational GC, and evacuation—all come together into one collector.